



Content retrieval strategy

- 1) Not blindly giving back content what LLM has to remember → 1 sec in a context window → LLM can't remember everything from start of our conversation so, make sure it's one of these strategies used for remembering content

Full context

- Recent messages only
- sliding window → messages / files
- summary → All entire messages into a single summary

task-state → preserve all completed tasks

Structured State → Only important information to be preserved

pruned context → All messages need to be made available

Based on them,

recent messages

+

Conversation summary

+

user preferences

+

current task state

+

memories

↓

LLM context

→ Retrieval Strategy:

What info should be kept
→ Given current task, which info
should we bring back

→ Planning a trip you had visited
once → you had problems
with that → so the
current task → you want
trip to be better now

Memory 1 → best when

2 → veg/nu-veg

3 → listen to this piece

like this

→ Clustering (one of the strategies for retrieval systems)

But not enough,

combine

→ Semantic clustering retrieval

→ Keyword clustering retrieval

→ Recency retrieval

→ Importance retrieval

Memory 2 → Entity retrieval

→ Metadata filtering

↓
top re-rank

↓
top memories

→ Pruning:
Memory storage can grow exponentially
10 memories
1000 memories
10,000 memories

Not everything needs to survive
Pruning → removing memories that
are not worth keeping

2+2 we need but prefers response in XML format

↳ preserved

So, the pruning system needs:

- ⇒ importance, frequency of use
- ⇒ recency, confidence
- ⇒ redundancy, expiration
- privacy.

→ Decay different from pruning.
gradually decrease / reduce the
importance / retrieval probability.

Useful
when
memories
temporary
preferences
changeable
or obsolete.

Is it more coding

Initially importance = 0.8

After years → 0.6 - 0.3

If I say I want to code more
then importance gain up ↑
Memory score importance

× recency ×

relevance ×

decaying

over time

confidence

→ Privacy and data boundaries

User

1. memories

Proj^A

Proj^B

work memories

now → Proj^A agent → not

retrieve imp

content of Proj^B

So memory should have
owner / permissions / retention policy
scope / sensitivity / source

→ Very imp in enterprise memories
agent types.

→ Session boundaries

Agent Able to correctly distinguish session
memory from LTM

like for now call this project as

Agentic AI memory

Project name =

Agent memory

Session memory
temp state

current plan

current memory

LTM

Past experience

preferences

long-term facts

Not every session detail is LTM